Practica 2

Felipe Murillo Gonzalez 1000135623
fmurillog@eafit.edu.co

Regresion simple: `https://youtu.be/ZLQPj2beUts`

Regresion multiple: `https://youtu.be/wwXLlVO8rLw`

Documentacion de metodos implementados

CSVReader::load. Carga un archivo CSV y lo descompone en una matriz de features `X` y un vector de etiquetas `y`. Recibe la ruta del archivo, un flag que indica si debe saltarse la cabecera, y referencias de salida para `X`, `y`, `nSamples` y `nFeatures`. Inicializa todas las salidas a `nullptr`/0, abre el archivo con `ifstream` (si falla imprime error a `cerr` y retorna), lee línea por línea con `getline`, salta líneas vacías y la cabecera si aplica, detecta el delimitador por línea (`;` o `,`), parsea cada token con `parseNumber` (que envuelve `stod` en `try/catch` y devuelve 0,0 en caso de error), guarda las filas temporalmente en un `vector<vector<double`, reserva `X = new double*[nSamples]` e `y = new double[nSamples]`, asigna la última columna a `y` y el resto a `X`, e imprime un resumen con las tres primeras filas. Retorna `void`; los datos se devuelven por referencia. Precondición: `filename` debe ser un CSV legible. Postcondición: o `X != nullptr` con datos válidos, o todo queda en `nullptr`/0 si la apertura falló.

CSVReader::freeData. Libera la memoria dinámica reservada por `load()`. Recibe la matriz `X`, el vector `y` y el número de muestras `nSamples`. Si `X != nullptr`, recorre $i = 0..nSamples - 1$ liberando cada fila con `delete[] X[i]`, luego libera el arreglo de punteros con `delete[] X`, y finalmente libera `y` con `delete[] y`. Retorna `void`. Precondición: `X`, `y` y `nSamples` deben coincidir con los devueltos por `load()`. Postcondición: toda la memoria queda liberada sin fugas. Es simétrica con `load()` y debe llamarse exactamente una vez por cada carga exitosa.

CSVReader::parseNumber. Convierte un token de texto a `double`, tolerando espacios y valores no numéricos. Recorta espacios, tabs y saltos de línea con `find_first_not_of` y `find_last_not_of`; si el token queda vacío retorna 0,0; intenta `stod(tok)` dentro de un `try/catch` y, si lanza excepción, retorna 0,0. Retorna el valor numérico o 0,0 si no se puede parsear. Es `static` (visibilidad de archivo), no forma parte de la interfaz pública.

RegressionModel::allocate (protegido). Reserva en el heap los tres arreglos del modelo. Asigna `nFeatures = n`, reserva `weights`, `mean` y `stddev` con `new double[n]()` (inicializados a cero), recorre `stddev` poniendo 1,0 en cada posición para que `applyScaling` funcione incluso antes de `data_scaling`, e inicializa `bias = 0.0` y `scaled = false`. Retorna `void`. Precondición: $n \geq 1$. Postcondición: los tres punteros apuntan a bloques válidos de n doubles. Se invoca solo desde el constructor base.

RegressionModel::deallocate (protegido). Libera los tres arreglos del modelo. Llama `delete[]` sobre `weights`, `mean` y `stddev`, y deja los tres punteros en `nullptr` para evitar doble liberación. Retorna `void`. Se invoca desde el destructor y desde `operator=` antes de copiar, garantizando que no queden bloques huérfanos al reasignar.

RegressionModel::copyFrom (protegido). Realiza copia profunda de todos los miembros desde otro modelo. Copia `nFeatures`, `bias` y `scaled`, reserva nuevos arreglos del tamaño correcto con `new double[nFeatures]` y copia elemento por elemento `weights`, `mean` y `stddev`. Retorna `void`. Precondición: `this` no debe tener arreglos válidos (o deben haberse liberado antes). Postcondición: `this` es una copia independiente de `o` sin aliasing. Lo usan el constructor copia y el `operator=`.

RegressionModel::RegressionModel(int n). Constructor que llama `allocate(n)` para reservar `weights`, `mean` y `stddev` con n elementos e inicializar `bias = 0`, `scaled = false` y `stddev[i] = 1`. Postcondición: objeto válido listo para que la subclase lo use.

RegressionModel::RegressionModel(const RegressionModel& o). Constructor copia que llama `copyFrom(o)` directamente (sin `allocate` previo, porque `copyFrom` ya reserva) para producir una copia profunda independiente del modelo origen, evitando que dos objetos compartan los mismos buffers.

RegressionModel::operator=. Operador de asignación con copia profunda. Verifica autoasignación con `this != &o`; si son distintos llama `deallocate()` para liberar lo actual y luego `copyFrom(o)`. Retorna `*this`. Postcondición: `this` es una copia independiente de `o`; si fue autoasignación no hace nada.

RegressionModel:: RegressionModel (destructor). Destructor virtual que llama `deallocate()`. La virtualidad permite que al destruir un objeto a través de un `RegressionModel*` se ejecute el destructor de la subclase correcta (necesario para `delete[]` del vector dinámico).

RegressionModel::score. Calcula MSE y R^2 del modelo sobre un conjunto de datos. Recibe la matriz `X` de predictores, el vector `y` de valores reales y el número de muestras `n`. Calcula primero la media de `y`, luego recorre las n muestras llamando `predict(X[i])` y acumula en `ssRes` la suma de $(y_i - \hat{y}_i)^2$ y en `ssTot` la suma de $(y_i - \bar{y})^2$; obtiene MSE = ssRes/n y $R^2 = 1 - \text{ssRes/ssTot}$ (o 0,0 si ssTot = 0), imprime ambos por `cout` y retorna el MSE. Precondición: $n > 0$. $R^2 = 1$ indica ajuste perfecto; $R^2 = 0$ indica que el modelo no supera a la media; $R^2 < 0$ puede ocurrir en test cuando la generalización es mala.

RegressionModel::data_scaling. Estandariza las columnas de `X` aplicando z-score $(x - \mu)/\sigma$ de forma in-place. Para cada columna calcula `mean[j]` como promedio de `X[i][j]` en una primera pasada; en una segunda acumula $(X[i][j] - \mu_j)^2$, divide entre n, aplica `sqrt` y guarda en `stddev[j]` con guardia `if (stddev[j] == 0.0) stddev[j] = 1.0`; finalmente transforma cada `X[i][j] = (X[i][j] - mean[j])/stddev[j]` y marca `scaled = true`. Precondición: $n > 0$. Postcondición: cada columna queda con media 0 y varianza 1, y los valores originales se pierden. Debe llamarse solo sobre train para evitar fuga de información; en OLS es opcional pero útil para estabilidad numérica.

RegressionModel::applyScaling. Devuelve una copia de `X` escalada con las μ, σ ya aprendidas por `data_scaling`, sin modificar la matriz original. Reserva `out = new double*[n]`, y para cada fila reserva `out[i] = new double[nFeatures]` y asigna `out[i][j] = (X[i][j] - mean[j])/stddev[j]`. Retorna `out`, propiedad del llamador que debe liberarlo con `delete[] out[i]` + `delete[] out`. Precondición: `data_scaling` debe haberse llamado antes. Pensado para el conjunto de test.

RegressionModel::print. Imprime por `cout` el `bias` y cada `weights[j]` con el formato configurado por el llamador (normalmente `setprecision(4)`). Retorna `void`. Útil para mostrar el resultado del entrenamiento en el reporte y el video.

RegressionModel::fit (virtual puro). Método virtual puro que entrena el modelo con los datos dados. No tiene implementación en la base; cada subclase define su algoritmo (OLS cerrado en simple, ecuación normal en múltiple). Recibe `X` de predictores, `y` de etiquetas y `n` de muestras.

Es el punto de polimorfismo del framework: el `main` llama `fit` a través de `RegressionModel*` sin conocer el tipo concreto.

RegressionModel::predict (virtual puro). Método virtual puro que predice la salida para una muestra `x` de `nFeatures` elementos. Sin implementación en la base; cada subclase calcula la predicción según su fórmula. Retorna `double`.

RegressionModel::modelName (virtual puro). Método virtual puro que devuelve una cadena identificando el tipo y método del modelo. Sin implementación en la base; cada subclase retorna un `const char*` descriptivo (por ejemplo `"SimpleLinearRegression (OLS cerrada)"`).

RegressionModel (getters inline). Accesores triviales definidos directamente en el header: `getWeights`, `getBias`, `getNumFeatures` e `isScaled`, que devuelven `weights`, `bias`, `nFeatures` y `scaled` respectivamente, sin lógica adicional, permitiendo consultar el estado del modelo desde fuera sin exponer los atributos como públicos.

SimpleLinearRegression::SimpleLinearRegression. Constructor que llama al constructor base con `RegressionModel(1)` para reservar `weights`, `mean` y `stddev` de tamaño 1. Postcondición: objeto válido listo para `fit`.

SimpleLinearRegression::fit. Ajusta $y = \text{bias} + w \cdot x$ por OLS cerrado. Primero calcula $\bar{x} = \text{mean}(X[i][0])$ e $\bar{y} = \text{mean}(y[i])$ en una pasada; luego acumula $\text{num} = \sum(X[i][0] - \bar{x})(y[i] - \bar{y})$ y $\text{den} = \sum(X[i][0] - \bar{x})^2$; asigna $w = \text{num}/\text{den}$ con guardia ($\text{den} \neq 0$) y $\text{bias} = \bar{y} - w \cdot \bar{x}$. Complejidad $O(n)$ en dos pasadas, sin iteraciones. No usa gradiente descendente. Postcondición: `weights[0]` y `bias` son la solución OLS exacta.

SimpleLinearRegression::predict. Predice el valor de y para una muestra `x` de un solo elemento, retornando $\text{bias} + \text{weights}[0] \cdot x[0]$.

SimpleLinearRegression::modelName. Identifica el modelo retornando `"SimpleLinearRegression (OLS cerrada)"`.

MultipleLinearRegression::MultipleLinearRegression. Constructor que llama a `RegressionModel(nFeatures)`. El valor por defecto 4 permite crear arreglos dinámicos con `new MultipleLinearRegression[N]`, ya que el compilador necesita un constructor invocable sin argumentos. Postcondición: objeto válido listo para `fit`.

MultipleLinearRegression::fit. Ajusta $y = \text{bias} + \sum_j w_j x_j$ resolviendo la ecuación normal $(X^\top X)\theta = X^\top y$ en cinco pasos: (1) construye $A = X^\top X$ y $\text{rhs} = X^\top y$ tratando la columna 0 como bias con valor 1; (2) aplica eliminación gaussiana con pivoteo parcial (intercambia filas según el mayor valor absoluto) y guardia if ($|A[col][col]| < 10^{-12}$) $A[col][col] = 10^{-8}$ para evitar división por cero; (3) sustitución hacia atrás para obtener θ; (4) extrae $\text{bias} = \theta[0]$ y $\text{weights}[j] = \theta[j+1]$; (5) libera temporales con `delete[]`. Complejidad $O(np^2 + p^3)$, trivial para $p = 4$. No usa gradiente descendente ni librerías externas. Postcondición: `weights` y `bias` son la solución OLS exacta o regularizada mínimamente si hubo singularidad.

MultipleLinearRegression::predict. Predice el valor de y para una muestra `x` de `nFeatures` elementos, inicializando $p = \text{bias}$ y acumulando $\text{weights}[j] \cdot x[j]$ para $j = 0..\text{nFeatures} - 1$; retorna el resultado.

MultipleLinearRegression::modelName. Identifica el modelo retornando `"MultipleLinearRegression (OLS - Ecuacion Normal)"`, dejando explícito que el método es OLS cerrado y no gradiente descendente.

Simplemain::runSimple. Ejecuta el flujo completo de la regresión simple. Carga el CSV con `CSVReader::load`, calcula $n_{\text{train}} = 0{,}8n$ y $n_{\text{test}} = n - n_{\text{train}}$, reserva `idx` con $0..n-1$ y lo baraja con `mt19937 rng(42)` para reproducibilidad, construye vistas reordenadas `Xs` e `ys` (sin copiar los

doubles, solo los punteros), entrena con `slr.fit(Xs, ys, nTrain)`, imprime pesos y bias con `print()`, evalúa con `score()` en TRAIN y TEST, hace un `predict()` de ejemplo comparando con el valor real, y libera todo simétricamente (`delete[] idx`, `delete[] Xs`, `delete[] ys`, `CSVReader::freeData`).

Simplemain::localScopeDemo. Crea un `SimpleLinearRegression localModel` en la pila e imprime su dirección (`&localModel`), su `sizeof` y el puntero `localModel.getWeights()` para ilustrar el segmento de memoria local en el análisis §3.2.

Simplemain::memoryAnalysis. Implementa el análisis de memoria de la sección §3.2 del PDF. Imprime `sizeof(SimpleLinearRegression)` con su desglose miembro por miembro (para responder al "why is that value?" del enunciado), imprime la dirección del `globalModel` (segmento `.data/.bss`) y su `weights`, llama `localScopeDemo()`, crea `new SimpleLinearRegression()` e imprime dirección y `getWeights()` antes de liberarlo con `delete`, crea `new SimpleLinearRegression[5]` mostrando $N \cdot$ sizeof, `&arr[0]` y las direcciones de cada elemento (56 B de separación $\rightarrow$ bloque contiguo), libera con `delete[]`, y finalmente imprime un bloque de texto explicando los segmentos y el hecho de que los buffers internos viven en heap independientemente del lugar del objeto.

Simplemain::main. Punto de entrada. Configura `fixed` y `setprecision(4)`, determina `path` a partir de `argv[1]` (o usa "Ïce_cream_selling_data.csv" por defecto), llama `runSimple(path)` y `memoryAnalysis()`, y retorna 0.

multiplemain::runMultiple. Flujo completo de la regresión múltiple. Idéntico en estructura a `runSimple()` pero usando `MultipleLinearRegression mlr(f)` donde f es el número real de features leídas del CSV (4 en `student_exam_scores.csv`). Carga, hace split 80/20, baraja con semilla 42, construye vistas reordenadas, entrena con OLS (sin estandarizar, para que los pesos queden en unidades originales interpretables), imprime `modelName()`, pesos y bias, evalúa en TRAIN y TEST con `score()`, hace un `predict()` de ejemplo, y libera simétricamente.

multiplemain::memoryAnalysisMultiple. Análisis de memoria §3.2 para la regresión múltiple, con estructura idéntica a `memoryAnalysis()`. Imprime `sizeof(MultipleLinearRegression) = 56 B` con su desglose (idéntico al de la base, ya que la subclase no añade miembros), las direcciones de `globalModel(4)` (en `.data/.bss`), `localModel(4)` (en pila) y `new MultipleLinearRegression(4)` (en heap) con sus respectivos punteros a `weights`, y crea `new MultipleLinearRegression[5]` mostrando $N \times \text{sizeof}(T) = 280$ B con las direcciones de cada elemento separadas 56 B. Aclara que aunque los buffers internos pesan $4 \times 8 = 32$ B cada uno en heap, el objeto sigue pesando 56 B porque solo guarda punteros.

multiplemain::main. Punto de entrada del programa de regresión múltiple. Configura `fixed` y `setprecision(4)`, asigna `path = (argc >= 2) ? argv[1] : "student_exam_scores.csv"`, llama `runMultiple(path)` y `memoryAnalysisMultiple()`, y retorna 0.

Inputs y outputs

Regresión simple

Archivo del código: `Simplemain.cpp`.

INPUT

Compilar en terminal:

```
g++ -std=c++11 -o regression.exe Simplemain.cpp RegressionModel.cpp
    SimpleLinearRegression.cpp CSVReader.cpp
```

Ejecutar:

```
regression.exe Ice_cream_selling_data.csv
```

OUTPUT

```
[CSVReader] Ice_cream_selling_data.csv -> 49 filas, 1 features
    fila 0: [-4.6623] -> y = 41.8430
    fila 1: [-4.3166] -> y = 34.6611
    fila 2: [-4.2140] -> y = 39.3830

  --- pesos aprendidos ---
  bias = 17.2442
  w[0] = -0.7827

  --- errores en TRAIN ---
  MSE = 153.4192
  R2  = 0.0290
  --- errores en TEST ---
  MSE = 107.7556
  R2  = -0.2403
  predict(X_test[0]) = 15.0279  (y real = 12.5682)

sizeof(SimpleLinearRegression) = 56 B

[Global]  &globalModel = 0x7ff6f8232040  | weights ptr = 0x19fd5d89210
[Local]   &localModel  = 0x57e53ffb10    | sizeof = 56 B
          weights ptr  = 0x19fd5dad940
[Dinamica] dyn         = 0x19fd5d78130   | weights ptr = 0x19fd5dadbb0

[VECTOR DINAMICO] new SimpleLinearRegression[5]
  N * sizeof(SLR) = 280 bytes
  Address of arr[0] = 0x19fd5d88098
    arr[0] @ 0x19fd5d88098 -> weights = 0x19fd5dad860
    arr[1] @ 0x19fd5d880d0 -> weights = 0x19fd5dad9a0
    arr[2] @ 0x19fd5d88108 -> weights = 0x19fd5dadb80
    arr[3] @ 0x19fd5d88140 -> weights = 0x19fd5dada90
    arr[4] @ 0x19fd5d88178 -> weights = 0x19fd5dad870
```

Interpretación. El R^2 de TRAIN es bajo (0,0290) y el de TEST es negativo ($-0{,}2403$), lo cual es coherente con la naturaleza de los datos: la relación entre temperatura y ventas de helado en `Ice_cream_selling_data.csv` es en forma de U (ventas altas tanto en frío como en calor, mínimas cerca de 0 °C), y un modelo lineal no puede capturar esa curvatura. No es un fallo del código, sino una limitación del modelo lineal frente a este conjunto de datos. Las direcciones de memoria mostradas son de una ejecución de ejemplo; en cada equipo cambian por ASLR, pero la separación de 56 B entre `arr[i]` y la ubicación de los `weights` en heap sí son invariantes.

Regresión múltiple

Archivo del código: `multiplemain.cpp`.

INPUT

Compilar en terminal:

```
g++ -std=c++11 -o multiple.exe multiplemain.cpp RegressionModel.cpp
    MultipleLinearRegression.cpp CSVReader.cpp
```

Ejecutar:

```
multiple.exe student_exam_scores.csv
```

OUTPUT

```
  [CSVReader] student_exam_scores.csv -> 200 filas, 4 features
    fila 0: [8.0000, 8.8000, 72.1000, 45.0000] -> y = 30.2000
    fila 1: [1.3000, 8.6000, 60.7000, 55.0000] -> y = 25.0000
    fila 2: [4.0000, 8.2000, 73.7000, 86.0000] -> y = 35.8000

  --- pesos aprendidos (unidades originales) ---
  bias = -2.2781
  w[0] = 1.5084
  w[1] = 0.9752
  w[2] = 0.1019
  w[3] = 0.1876

  --- errores en TRAIN ---
  MSE = 6.8684
  R2  = 0.8520
  --- errores en TEST ---
  MSE = 9.1738
  R2  = 0.7804
  predict(X_test[0]) = 38.8539  (y real = 39.5000)

sizeof(MultipleLinearRegression) = 56 B

[GLOBAL]   &globalModel = 0x7ff7cf163040 | weights ptr = 0x1b2961ddcd0
           (segmento .data/.bss)
[LOCAL]    &localModel  = 0xd3c3ffca0   | weights ptr = 0x1b2961de240
           (pila / stack)
[DINAMICO] dyn          = 0x1b2961c75b0 | weights ptr = 0x1b2961de3c0
           (heap)

[VECTOR DINAMICO] new MultipleLinearRegression[5]
  N * sizeof(MLR) = 5 * 56 = 280 bytes
  Address of arr[0] = 0x1b2961d9e48
  (bloque CONTIGUO de N objetos en heap + array cookie)
    arr[0] @ 0x1b2961d9e48 -> weights = 0x1b2961de030
    arr[1] @ 0x1b2961d9e80 -> weights = 0x1b2961de2a0
    arr[2] @ 0x1b2961d9eb8 -> weights = 0x1b2961de0f0
    arr[3] @ 0x1b2961d9ef0 -> weights = 0x1b2961ddf70
    arr[4] @ 0x1b2961d9f28 -> weights = 0x1b2961de360
```

Interpretación. El modelo alcanza $R^2 = 0{,}8520$ en TRAIN y $R^2 = 0{,}7804$ en TEST, valores altos y consistentes que indican que el modelo generaliza bien sin sobreajuste severo. Los pesos quedan en unidades originales: `w[0]` = 1,5084 puntos de `exam_score` por hora adicional de estudio, `w[1]` = 0,9752 puntos por hora de sueño, `w[2]` = 0,1019 puntos por 1 % de asistencia, `w[3]` = 0,1876 puntos por punto de calificación previa, y el `bias` = −2,2781 es el valor base cuando todas las

features son 0. La predicción de ejemplo (38,8539 frente al valor real 39,5000) confirma un error bajo. Las direcciones de memoria cambian por equipo y por corrida por ASLR, pero la separación de 56 B entre `arr[i]` y la ubicación de los `weights` en heap son invariantes.

Problemas y soluciones

Gradiente descendente vs. solución cerrada. La primera versión de `MultipleLinearRegression::fit` usaba gradiente descendente con hiperparámetros `learningRate` y `epochs`, porque es el algoritmo que suele asociarse intuitivamente con "entrenar un modelo". Sin embargo, para regresión lineal el método correcto y óptimo es la solución cerrada por ecuación normal $(X^\top X)\theta = X^\top y$, que no requiere hiperparámetros y da el mínimo exacto de la suma de residuos al cuadrado. Se resolvió reescribiendo `fit` para construir $A = X^\top X$ y rhs $= X^\top y$ y resolver el sistema con eliminación gaussiana y pivoteo parcial, eliminando por completo `learningRate` y `epochs` del flujo efectivo. El R^2 obtenido resultó igual o ligeramente mejor que el del gradiente, con la ventaja adicional de ser determinista.

Detección del delimitador del CSV. Durante las pruebas surgió un problema cuando el archivo `student_exam_scores.csv` tenía como delimitador el punto y coma (`;`) en lugar de la coma (`,`). El lector original solo dividía por coma, de modo que cada línea quedaba con una sola columna, la matriz `X` tenía dimensiones incorrectas y las predicciones fallaban silenciosamente. La solución fue modificar `CSVReader::load` para que detecte el delimitador por línea: si la línea contiene `;`, usa `;`; en caso contrario, usa `,`. Esto hace que el mismo binario sirva para ambos archivos sin cambios ni conversiones manuales.

Compilación de proyectos multi-archivo en VS Code. Al usar el botón de compilación de VS Code, la tarea por defecto de `tasks.json` compilaba únicamente el archivo activo (`${file}`), lo que producía errores de enlazado del tipo `undefined reference to MultipleLinearRegression::fit`. No era un error del código, sino de la configuración del editor. La solución fue reemplazar `tasks.json` por dos tareas que compilan explícitamente todos los `.cpp` necesarios (`multiplemain.cpp`, `RegressionModel.cpp`, `MultipleLinearRegression.cpp`, `CSVReader.cpp`) y añadir una segunda tarea para el binario de regresión simple. Como alternativa, se documentó la compilación por terminal con el comando `g++ -std=c++11 -o multiple.exe ...`, que no depende del editor.

Conflicto por múltiples funciones `main`. El archivo `test.cpp` que se usó para verificar la instalación del compilador contenía su propio `int main()`. Al incluir por error todos los `.cpp` de la carpeta en un mismo comando de compilación, el enlazador fallaba con `multiple definition of 'main'`. La solución fue excluir `test.cpp` de los comandos de compilación de los binarios `regression.exe` y `multiple.exe`, y documentar en el reporte que ese archivo es solo de prueba y no forma parte del proyecto entregado.

Gestión de memoria dinámica en las copias. La clase base `RegressionModel` reserva tres arreglos en el heap (`weights`, `mean`, `stddev`). Si el compilador generaba el constructor copia por defecto, dos objetos apuntarían a los mismos buffers y el destructor haría doble `delete[]`, causando corrupción de memoria o caída del programa. Se resolvió implementando la regla de los cuatro: constructor copia que llama a `copyFrom`, `operator=` que verifica auto-asignación y libera antes de copiar, y destructor virtual que llama a `deallocate`. Esto garantiza copias profundas e independientes y elimina las fugas cuando un objeto se reasigna.

Texto que se sale del margen en LaTeX. Al documentar los métodos con identificadores largos como `MultipleLinearRegression::memoryAnalysisMultiple` o `vector<vector<double`, LaTeX no podía partirlos porque `\texttt` no es guionable por defecto, y las líneas se desbordaban del margen. Se solucionó cargando el paquete `[htt]{hyphenat}`, que permite partir palabras

dentro de \texttt, junto con microtype y \sloppy para estirar los espacios cuando el corte no es posible. Con esto el PDF compila sin advertencias de Overfull \hbox.

Conclusiones

Conclusión 1. Para regresión lineal, la solución cerrada por ecuación normal es preferible al gradiente descendente: es determinista, no requiere hiperparámetros y entrega el óptimo exacto de la suma de residuos al cuadrado. El gradiente descendente solo se justifica cuando el sistema es demasiado grande para invertir la matriz $X^\top X$ o cuando el modelo no es lineal, condiciones que no aplican en esta práctica con cuatro features y doscientos registros.

Conclusión 2. La estandarización de las variables independientes no altera el R^2 ni el MSE en OLS, porque el modelo es invariante a transformaciones afines de X; solo cambia la escala de los coeficientes. Por eso se dejó el flujo sin escalado: los pesos quedan directamente interpretables en las unidades del problema (w[0] = puntos de exam_score por hora de estudio, w[2] = puntos por 1 % de asistencia, etc.), lo cual es más útil para el reporte. El método data_scaling() está implementado en el framework porque el enunciado lo exige, y su efecto fue verificado experimentalmente.

Conclusión 3. El análisis de memoria confirma que sizeof(SimpleLinearRegression) y sizeof(MultipleLinearRegression) valen ambos 56 B, no porque las subclases añadan datos, sino porque heredan el mismo layout de RegressionModel: un vptr de 8 B por los métodos virtuales, cuatro miembros de 8 B (weights, bias, mean, stddev), un int de 4 B, un bool de 1 B y padding para alinear a 8 B. Los arreglos internos no viven dentro del objeto: se reservan aparte con new[] en allocate(), por eso cada modelo tiene múltiples direcciones de heap asociadas y los buffers cambian de tamaño según nFeatures mientras el objeto sigue midiendo 56 B.

Conclusión 4. El uso de un framework orientado a objetos con una clase base abstracta (RegressionModel) y dos subclases concretas (SimpleLinearRegression, MultipleLinearRegression) demuestra las ventajas del polimorfismo: score, data_scaling, applyScaling y print se implementan una sola vez en la base y funcionan para ambos modelos, mientras que solo fit, predict y modelName se especializan. Además, la regla de los cuatro (constructor, constructor copia, operator= y destructor virtual) es indispensable cuando la clase posee punteros con new[], porque evita el doble delete y las fugas de memoria al copiar o reasignar objetos.

Conclusión 5. El R^2 bajo de la regresión simple (0,0290 en TRAIN y −0,2403 en TEST) no es un defecto del código ni del algoritmo OLS, sino una limitación del modelo lineal frente a un conjunto de datos cuya relación es en forma de U (ventas de helado altas tanto en temperaturas muy frías como muy cálidas, mínimas cerca de 0 °C). Esto motiva la extensión natural del framework a regresión polinómica añadiendo el término x^2 como feature adicional, reutilizando la clase MultipleLinearRegression con dos features. La conclusión práctica es que el R^2 de TRAIN y TEST debe interpretarse siempre a la luz de la forma de los datos y del modelo, y que reportar solo TRAIN oculta problemas de generalización que el R^2 de TEST sí revela.

Conclusión 6. Las direcciones de memoria impresas por el análisis de memoria varían entre ejecuciones y entre equipos debido al Address Space Layout Randomization (ASLR), que aleatoriza la base de los segmentos .data, pila y heap en cada corrida. Lo que permanece invariante es la relación estructural entre las direcciones: arr[i] dista exactamente 56 B del siguiente, los weights de cada objeto viven en heap aunque el objeto esté en pila o en .data, y sizeof(T) es constante. Comprender esta distinción es clave al documentar análisis de memoria en el reporte, porque los valores absolutos son ejemplos, no contratos.

Fallos de software por gestion de memoria dinamica

Reinicio del Mars Pathfinder (1997). La misión Mars Pathfinder fue un proyecto de la NASA gestionado por el Jet Propulsion Laboratory (JPL), con software de vuelo corriendo sobre el sistema operativo de tiempo real VxWorks de Wind River Systems; el fallo fue diagnosticado por el ingeniero del JPL David Gruel y explicado públicamente por David Wilner, director de tecnología de Wind River, en el IEEE Real-Time Systems Symposium de 1997. Pocos días después del aterrizaje del 4 de julio de 1997, la nave comenzó a experimentar reinicios totales intermitentes que provocaban pérdida de datos científicos y meteorológicos; el reinicio era en realidad un watchdog timer que se disparaba cuando una tarea crítica no se ejecutaba a tiempo. La causa raíz fue una *inversión de prioridad* en el manejo de memoria compartida: una tarea de baja prioridad tomaba un mutex para escribir en el bus de información, una tarea de alta prioridad quedaba bloqueada esperando ese mutex, y una tarea de prioridad media se adelantaba impidiendo que la de baja prioridad liberara el recurso. La solución fue habilitar la *herencia de prioridad* en el mutex de VxWorks: el mecanismo ya existía, pero el parámetro booleano que lo activaba estaba en `false`; bastó cambiarlo a `true` y reprogramar la nave desde tierra para corregirlo de forma definitiva.

Caída de Amazon Web Services por fuga de memoria (2012). Amazon Web Services (AWS), la división de servicios en la nube de Amazon, sufrió el 22 de octubre de 2012 una interrupción de más de 12 horas en su región `US-East-1` (Norte de Virginia) que afectó a sitios como Reddit, Foursquare, Minecraft, Heroku, GitHub, Imgur, Pocket, HipChat, Coursera, Airbnb y Flipboard, entre otros; AWS emitió reembolsos a los clientes afectados y tuvo que explicar públicamente el incidente en un post-mortem. La causa raíz fue una fuga de memoria latente: tras sustituir un servidor de recogida de datos defectuoso, el nuevo servidor no propagó correctamente su dirección DNS; una fracción de los agentes de monitorización siguió intentando conectarse al servidor eliminado y, en lugar de manejar el fallo de conexión de forma controlada, reintentaba la conexión en un bucle que consumía memoria gradualmente. La fuga pasó desapercibida porque las alarmas no estaban configuradas para detectar ese patrón específico de consumo de memoria en un sistema que por diseño usa memoria de forma dinámica y fluctuante, de modo que el servicio EBS dejó de procesar solicitudes y los volúmenes se atascaron, propagando la degradación a RDS y a los Elastic Load Balancers. La corrección consistió en añadir alarmas específicas basadas en tendencias (crecimiento sostenido) y no solo en umbrales absolutos, corregir el agente de recogida de datos para que ante una conexión fallida liberara los recursos en lugar de reintentar indefinidamente, y mejorar la monitorización general de memoria de EBS que por su naturaleza dinámica no tenía umbrales bien definidos.

Fuga de memoria en Atlassian Confluence (2023). Atlassian, la empresa australiana desarrolladora de Confluence, Jira y otras herramientas de colaboración, sufrió el 14 de junio de 2023 una inestabilidad de aproximadamente 55 minutos (entre las 18:55 y las 19:50 UTC) que afectó a clientes de Confluence Cloud en las regiones `prod-west2` y `prod-east` de Estados Unidos, publicada por el equipo de Customer Support en el post-mortem oficial. El impacto no fue uniforme: algunas peticiones de algunos usuarios se completaban y otras fallaban, dependiendo de qué nodo del gateway atendiera la solicitud, y el servicio se restableció escalando horizontalmente el gateway de API de Confluence y revirtiendo el cambio de nombre que había desencadenado el fallo. La causa raíz fue un cambio de nombre de un servicio interno de infraestructura de monitorización que Confluence invocaba desde el gateway en *cada petición web*; tras el renombrado, el gateway empezó a fallar en esa invocación y, en lugar de manejar el error de forma controlada, entró en un estado que provocaba una fuga de memoria, agotando bajo carga alta la memoria disponible de los hosts y dejando el servicio inaccesible. El fallo no se detectó en *staging* porque con carga baja la fuga era muy lenta y con carga alta las pruebas de carga activas enmascaraban los fallos de nodo. La corrección consistió en que el gateway usara *variables de entorno* para

invocar servicios de infraestructura en lugar de nombres codificados, en hacer el código del gateway resiliente a la indisponibilidad de ese servicio no crítico permitiendo *bypass* en lugar de fuga, en añadir una verificación en tiempo de despliegue que fallara si una dependencia clave no se podía invocar, en verificar que un servicio desmantelado no siguiera en uso antes de retirarlo, y en activar alarmas de memoria del gateway también en *staging* con menos retardo.

Referencias

[1] Atlassian Status Page. Atlassian's cross product user search service is currently degraded. Atlassian Status Page, December 2023.

[2] Atlassian Status Page. Confluence instability — incident report. Atlassian Status Page, June 2023.

[3] Mike Jones. What really happened on mars rover pathfinder. Risk Forum, December 1997.

[4] Network World. Amazon cites cause of recent outage, issues refunds. Network World, November 2012.

[5] Raj Rajkumar. What happened on mars? Carnegie Mellon University, December 1997.

[6] Lui Sha, Ragunathan Rajkumar, and John P. Lehoczky. Priority inheritance protocols: An approach to real-time synchronization. *IEEE Transactions on Computers*, 39(9):1175–1185, 1990.

[7] TechCrunch. Amazon web services outage caused by memory leak and failure in monitoring alarm. TechCrunch, October 2012.

[8] The New York Times. Problem stalling mars study is reported solved. *The New York Times*, July 1997.

[9] Wikipedia. Amazon web services. Wikipedia, The Free Encyclopedia.

[2] [1] [3] [4] [8] [5] [6] [7] [9]